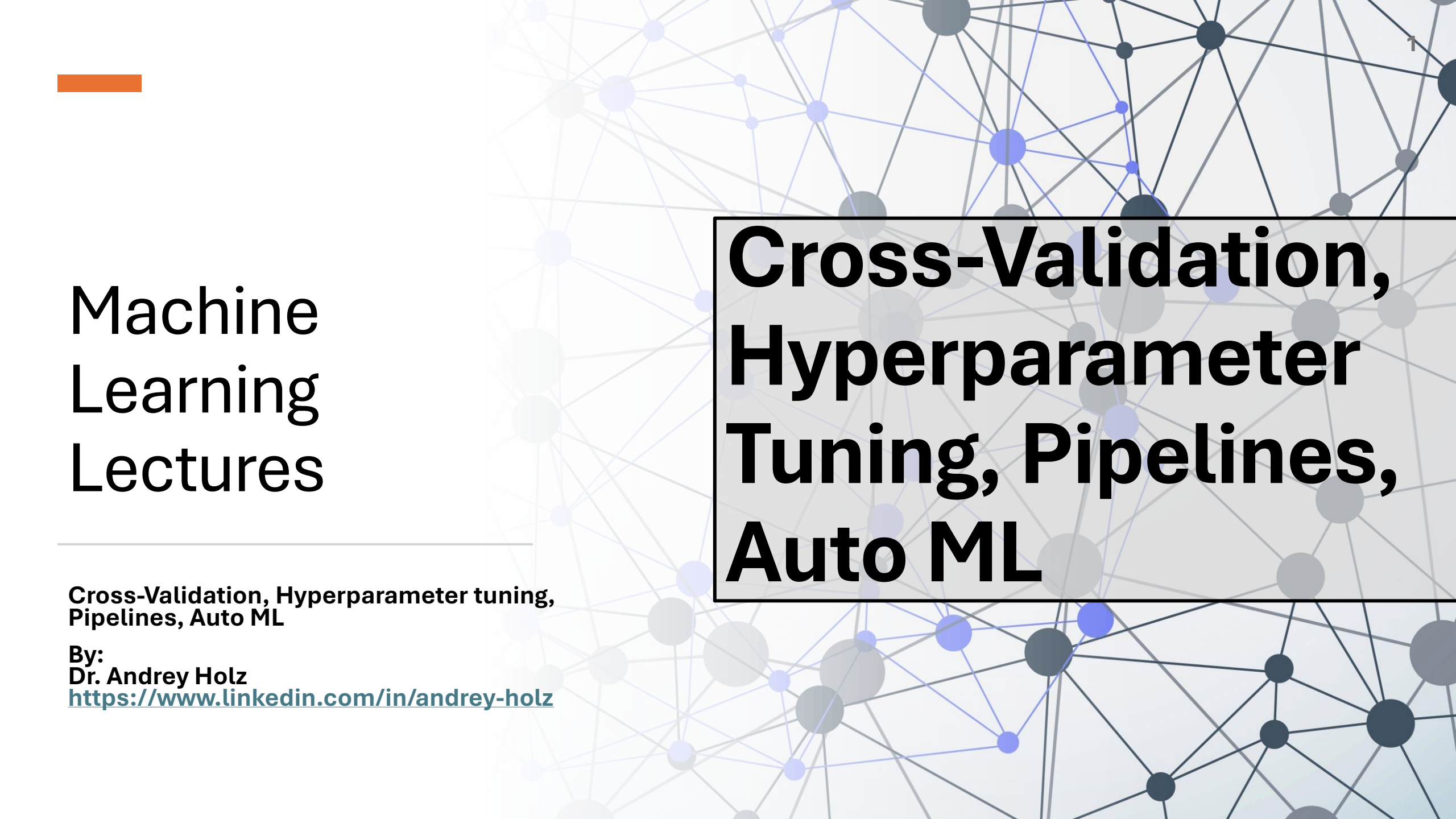




Machine Learning Lectures

Cross-Validation, Hyperparameter tuning,
Pipelines, Auto ML

By:
Dr. Andrey Holz
<https://www.linkedin.com/in/andrey-holz>



Cross-Validation, Hyperparameter Tuning, Pipelines, Auto ML



Machine Learning Lectures

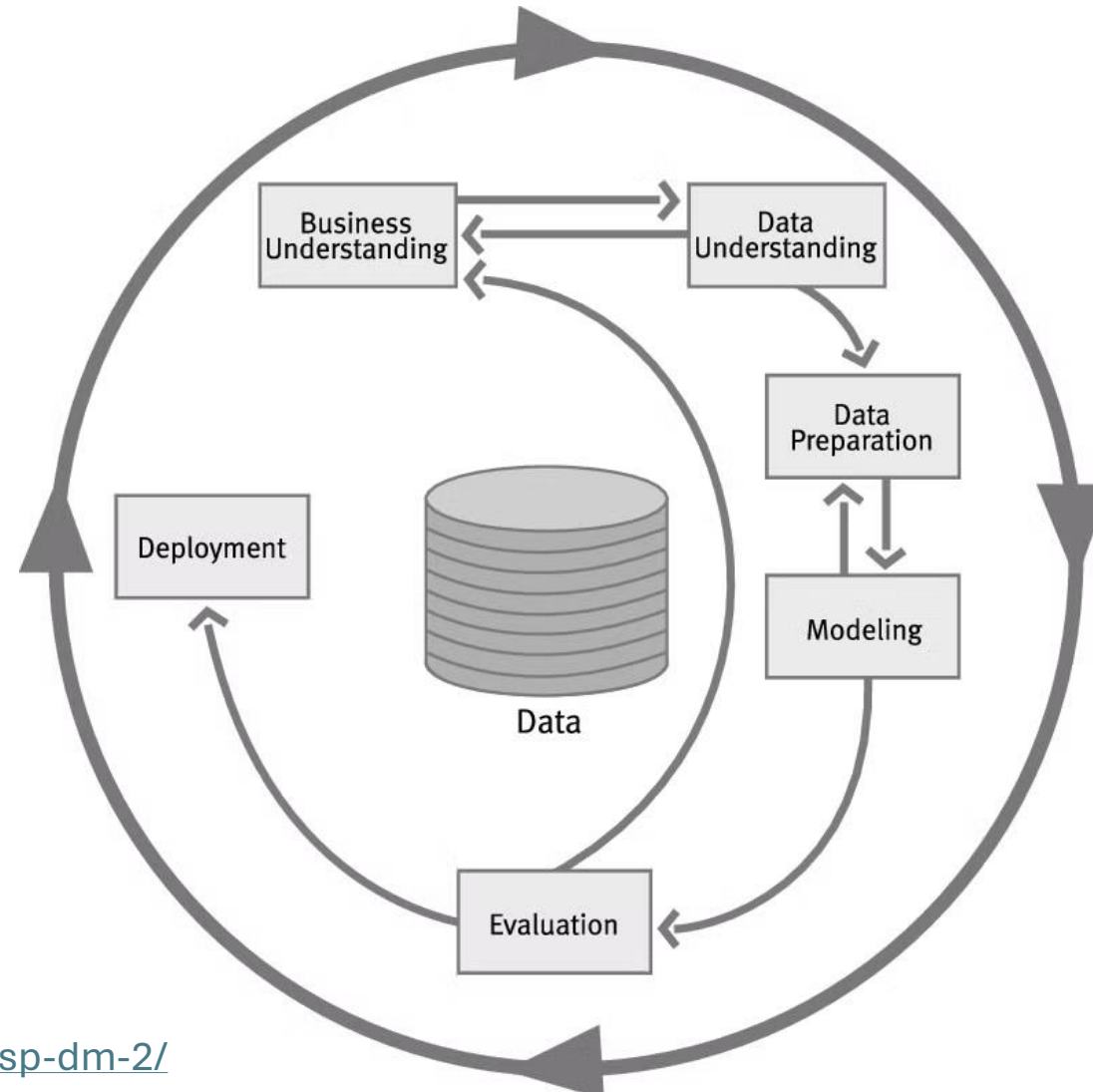
**Cross-Validation, Hyperparameter tuning,
Pipelines, Auto ML**

By:
Dr. Andrey Holz
<https://www.linkedin.com/in/andrey-holz>

Agenda

1. Cross-Validation
2. Hyperparameter Search
3. ML Pipelines
4. Introduction to AutoML
5. Limitations and Considerations
6. Key Takeaway

Data Analytics pipeline: CRISP-DM



<https://www.datascience-pm.com/crisp-dm-2/>

<https://web.archive.org/web/20220401041957/https://www.the-modeling-agency.com/crisp-dm.pdf>

Business Understanding	Data Understanding	Data Preparation	Modeling	Evaluation	Deployment
Determine Business Objectives Background Business Objectives Business Success Criteria Assess Situation Inventory of Resources Requirements, Assumptions, and Constraints Risks and Contingencies Terminology Costs and Benefits Determine Data Mining Goals Data Mining Goals Data Mining Success Criteria Produce Project Plan Project Plan Initial Assessment of Tools and Techniques	Collect Initial Data Initial Data Collection Report Describe Data Data Description Report Explore Data Data Exploration Report Verify Data Quality Data Quality Report	Select Data Rationale for Inclusion/Exclusion Clean Data Data Cleaning Report Construct Data Derived Attributes Generated Records Integrate Data Merged Data Format Data Reformatted Data Dataset Dataset Description	Select Modeling Techniques Modeling Technique Modeling Assumptions Generate Test Design Test Design Build Model Parameter Settings Models Model Descriptions Assess Model Model Assessment Revised Parameter Settings	Evaluate Results Assessment of Data Mining Results w.r.t. Business Success Criteria Approved Models Review Process Review of Process Determine Next Steps List of Possible Actions Decision	Plan Deployment Deployment Plan Plan Monitoring and Maintenance Monitoring and Maintenance Plan Produce Final Report Final Report Final Presentation Review Project Experience Documentation

Why Does Model Validation & Tuning Matter?

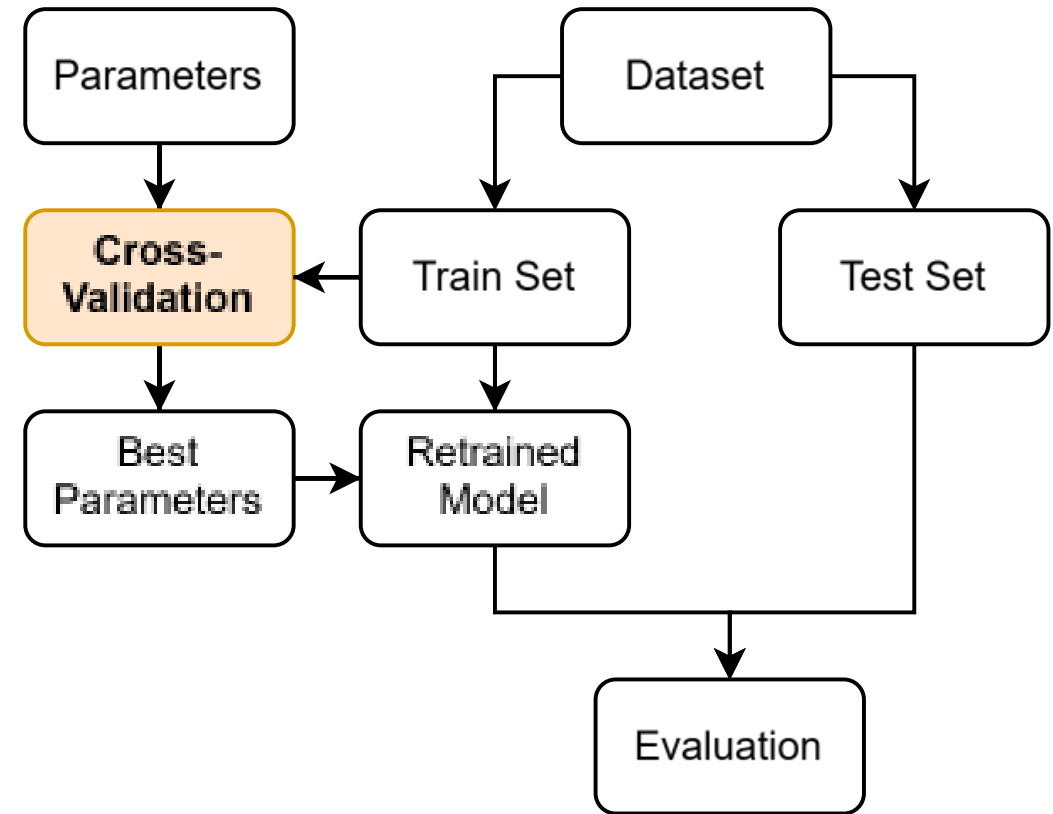
- **Overfitting:**
Without proper validation, models may memorize training data.
- **Generalization:**
Good performance on unseen data requires systematic validation.
- **Efficiency:**
Prevent wasted computational resources on poor modeling strategies.



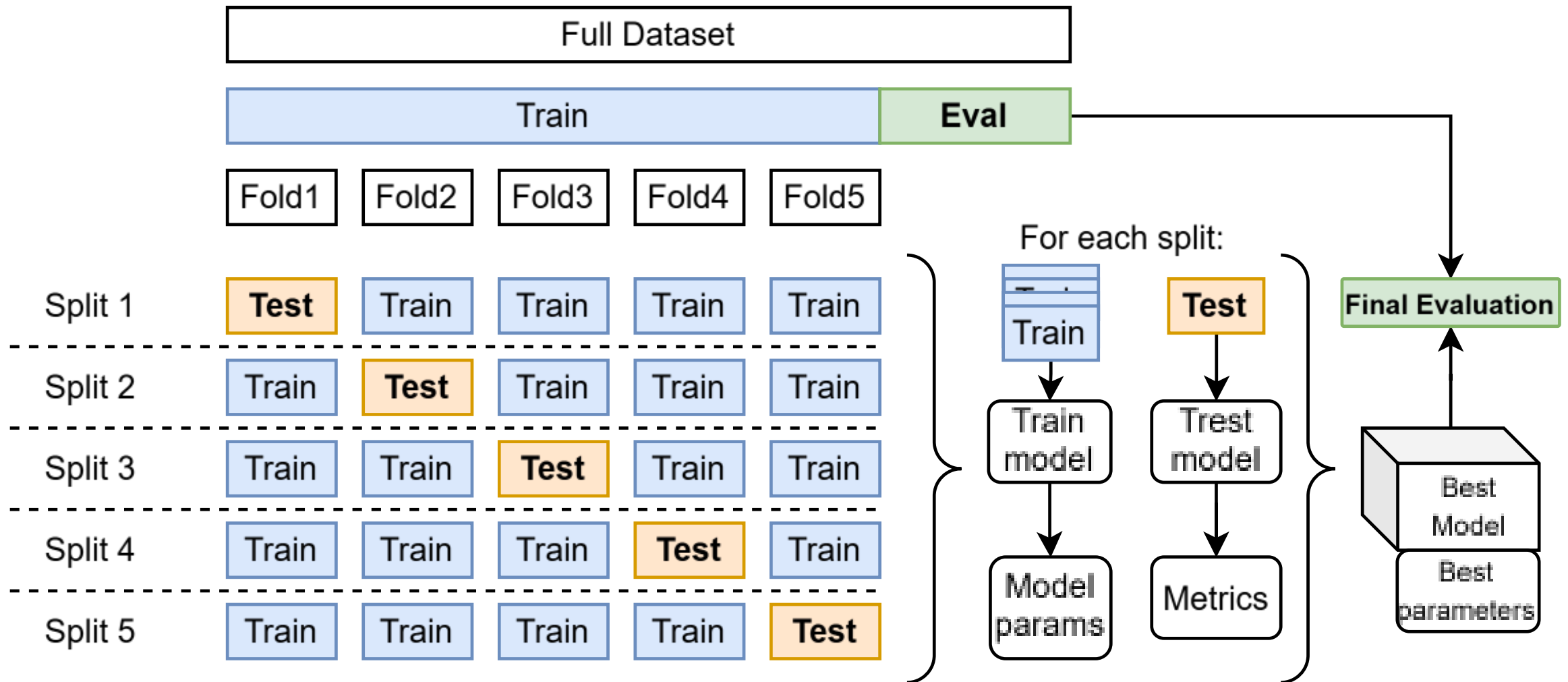
Introduction to Cross- Validation

Cross-Validation Basics

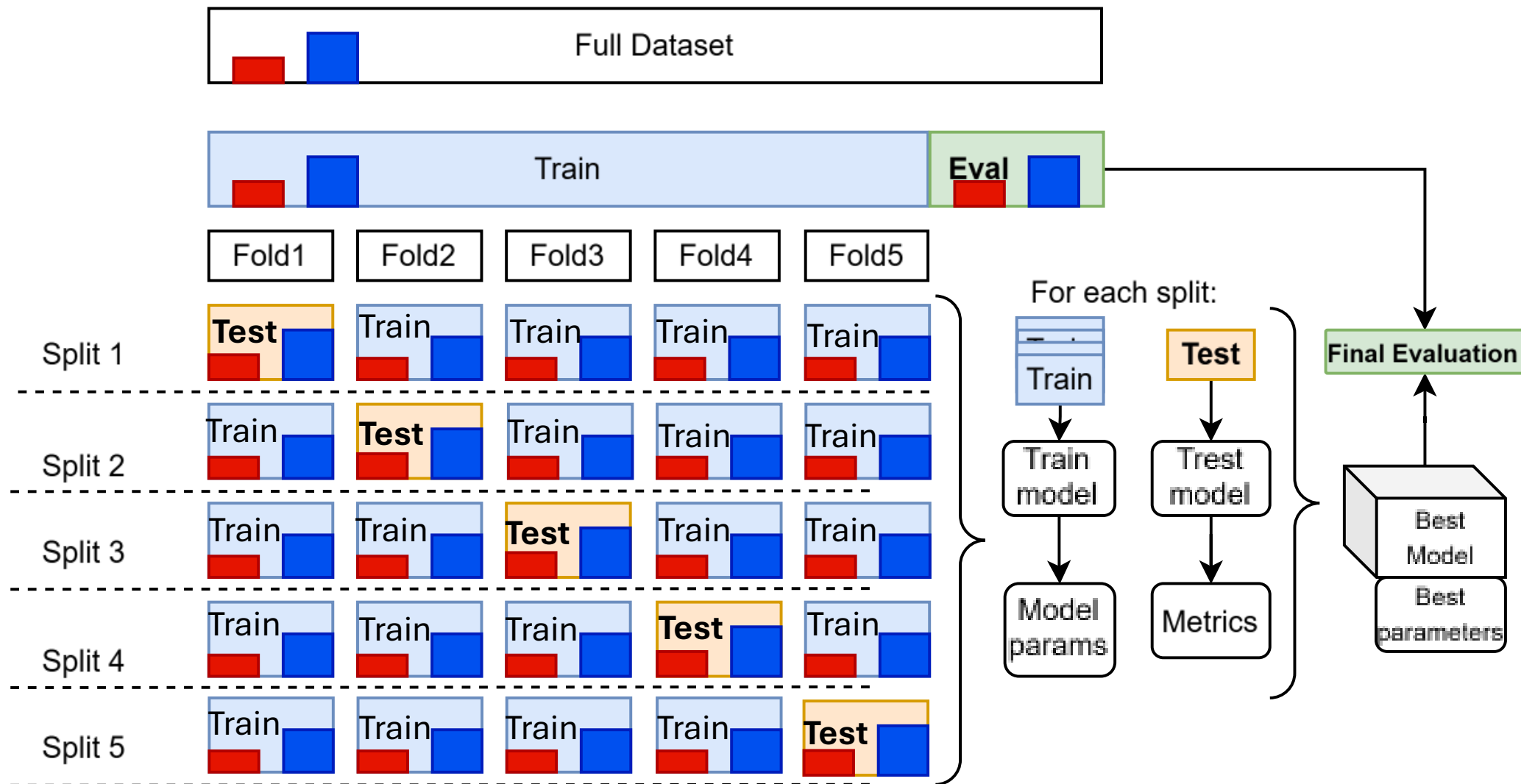
- **What is Cross-Validation (CV)?**
A method to systematically split data into training and validation sets.
- **Goals:**
 - Obtain stable performance metrics.
 - Better utilize limited data.
- **Common CV strategies:**
 - **K-Fold** (e.g., 5 or 10 folds)
 - **Stratified K-Fold** (for imbalanced data)
 - **Leave-One-Out CV** (to utilize all data)



K-Fold cross validation (K=5)

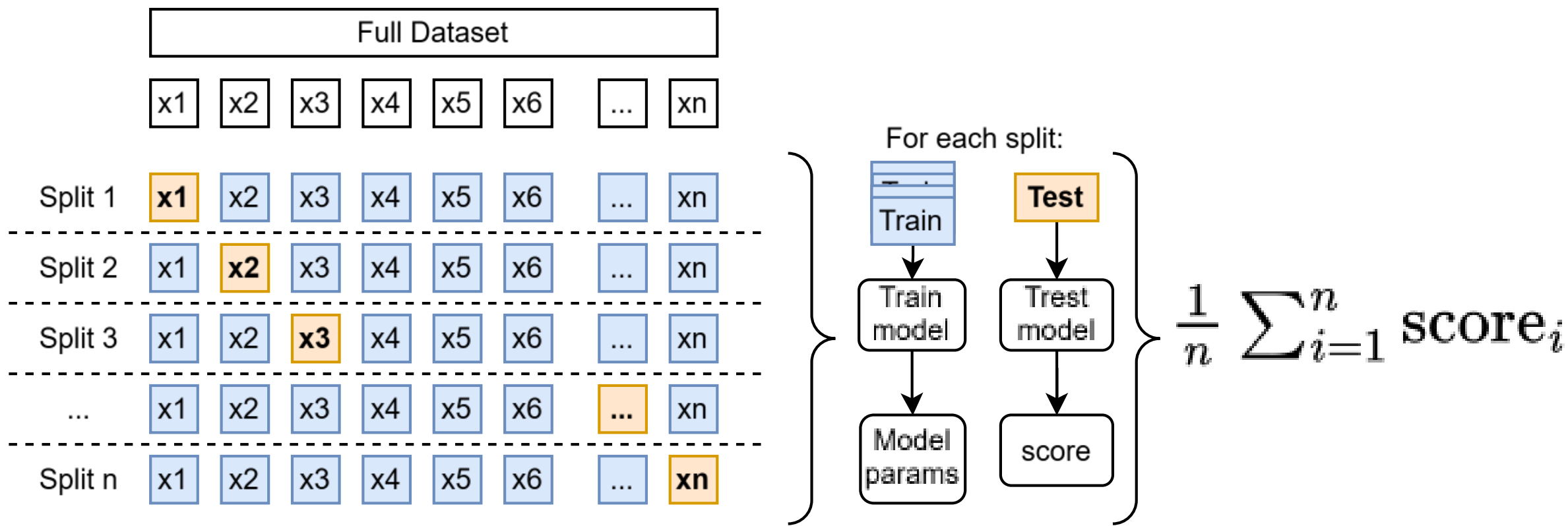


Stratified K fold



Leave-One-Out Cross Validation

- Extreme case of K-Fold

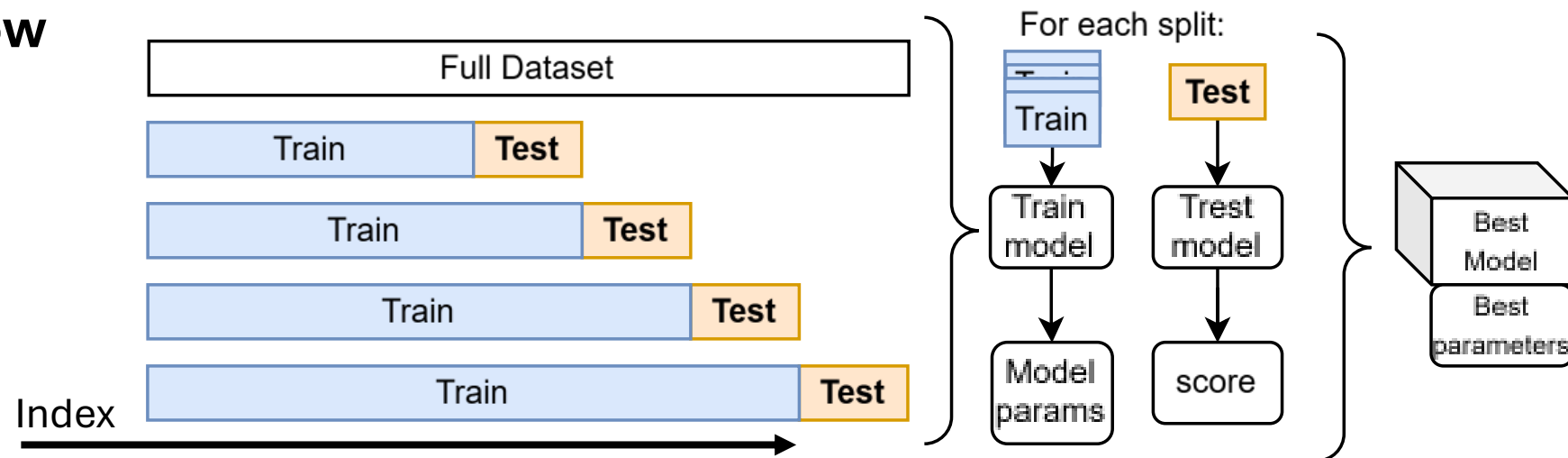


Advanced CV Techniques

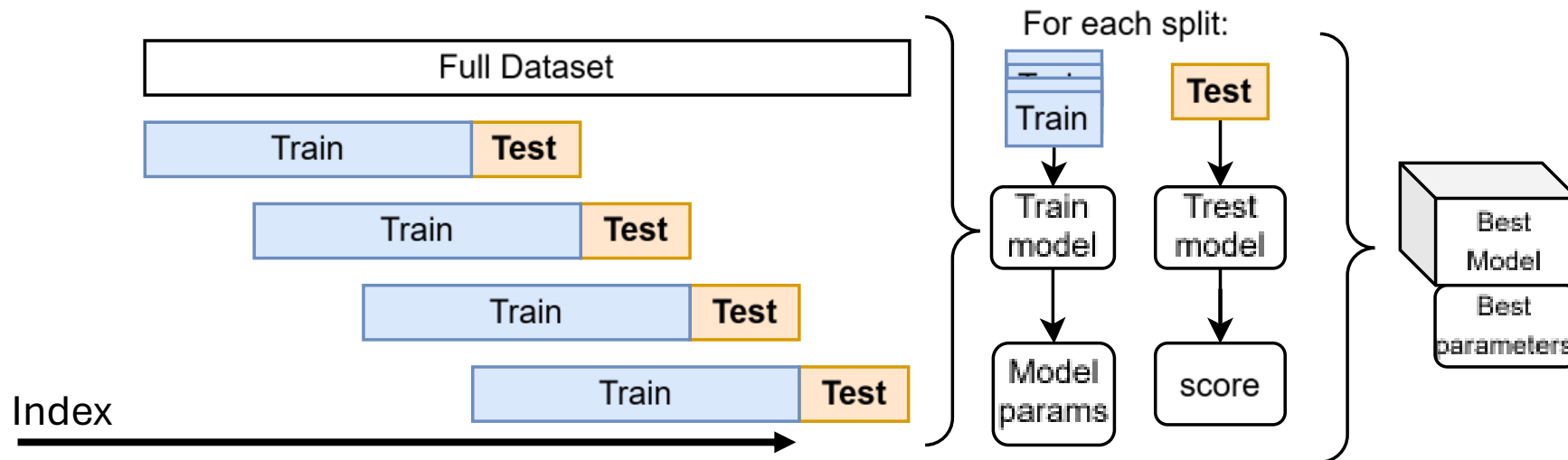
- **Time Series Split** for chronological data
- **Nested Cross-Validation** for unbiased hyperparameter tuning comparisons
- **Repeated K-Fold** to reduce variance

Time Series Cross Validation

Expanding Window



Sliding Window



Nested Cross-Validation

Goal: Unbiased performance estimation when tuning hyperparameters

Structure: 2 loops

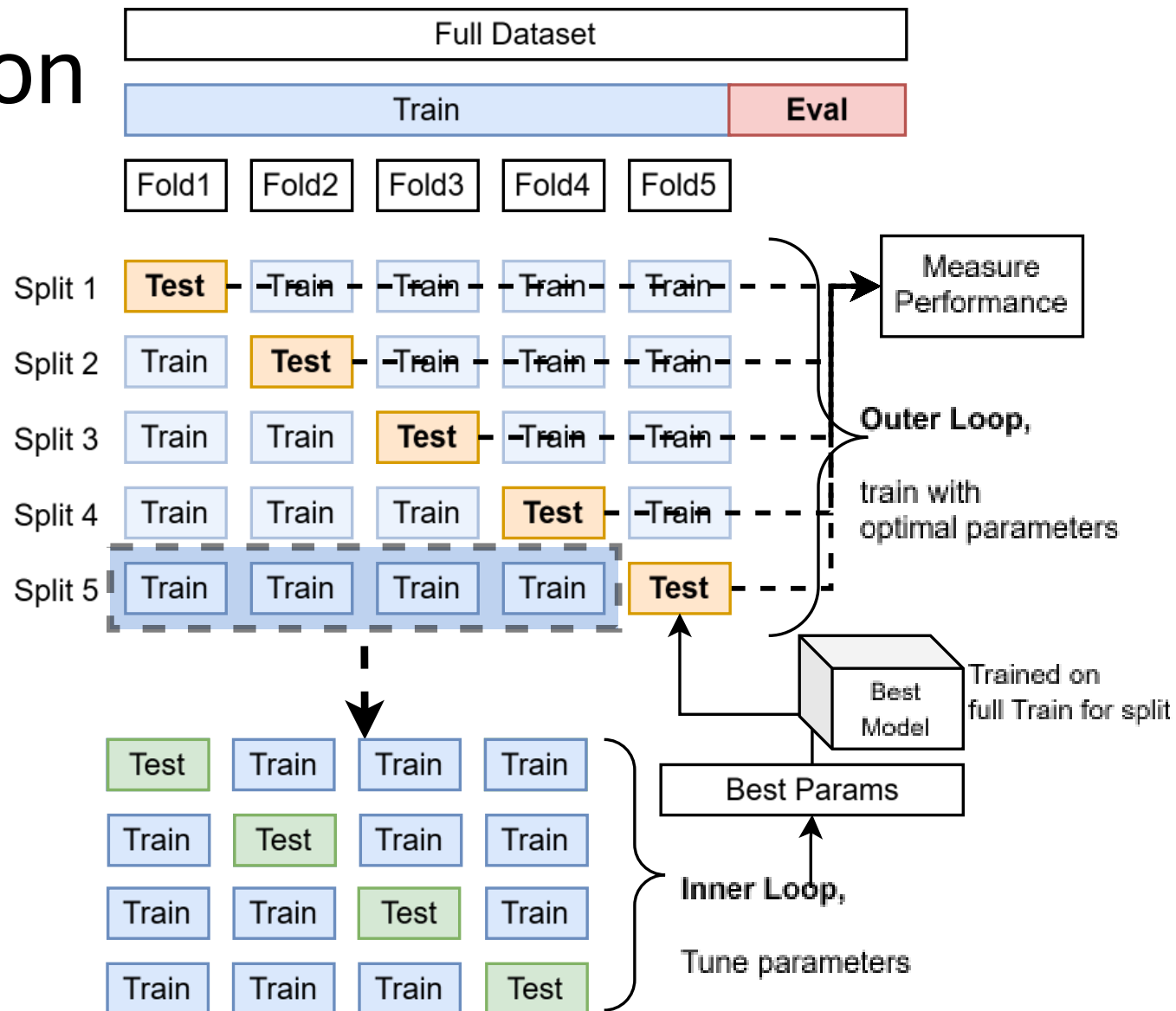
- **Outer Loop:** Splits data into training and test folds for final performance evaluation
- **Inner Loop:** Performs hyperparameter tuning (e.g., GridSearchCV) on the training folds only

Why It Matters

- Prevents **overly optimistic** results
- Separates **model selection** (inner loop) from **model evaluation** (outer loop)

Trade-Off

- Increased **computational cost** (cross-validation *within* cross-validation)



Repeated K-Fold

Runs K-Fold Cross-Validation multiple times with **different random splits** each iteration.

Purpose

- **Reduce variance** in performance estimates by averaging scores over multiple runs.
- Helps mitigate issues from a single, potentially unrepresentative partitioning.

Process:

1. **K-Fold**: Split dataset into K folds, train on K-1 folds, and validate on 1 fold—repeat until each fold is used as the validation set.
2. **Repeat**: Shuffle and re-split the data again (up to N times), then average all results.

Considerations

- Increases **computational cost** (K-Fold times number of repeats).
- More **robust** estimation, especially for smaller datasets.



Cross Validation for non-standard cases

Multiclass Cross-Validation

The dataset often contains imbalanced class distributions, meaning certain classes may have significantly fewer examples than others.

Standard K-Fold CV may lead to folds where some classes are underrepresented or missing entirely, resulting in unreliable performance estimates.

Strategies:

- **Stratified K-Fold CV**
 - Preserves the class distribution across folds.
 - Works well for multiclass classification by ensuring each fold has a representative proportion of all classes.
 - Example: If class A makes up 50%, class B 30%, and class C 20%, each fold will maintain this distribution.
- **Leave-One-Group-Out CV (Optional)**

When you have group-dependent data (e.g., patients, experiments), this ensures each group appears only in the validation set once.

Multilabel Cross-Validation

Each sample can belong to multiple classes. Standard stratified CV doesn't directly apply.

Iterative Stratification:

- Maintains label distributions across folds.
- Helpful in datasets with high label correlations.

Binary Relevance Approach:

- Treat each label as an independent binary classification problem.
- Apply stratified CV per label, then aggregate results.

Evaluation Metrics:

- Hamming Loss: Fraction of misclassified labels.
- Micro/Macro-Averaged F1: Combines performance across multiple labels.

Regression Cross-Validation

Standard K-Fold

- Common default method for i.i.d. data.
- Can be sensitive to outliers or skewed target distributions.

Stratified Approach (via Binning)

- Discretize the continuous target into bins, then apply Stratified K-Fold.
- Balances target distribution across folds.

Time Series Split

- For **temporal data**, ensures training always precedes validation in time.
- Prevents data leakage from the future into the past.

Metrics:

- **MSE / RMSE**: Penalize large errors more heavily.
- **MAE**: More robust to outliers.
- **R² Score**: Proportion of variance explained by the model.

CROSS-VALIDATION
SUMMARY

Cross-Validation Summary

Technique	Ideal Use Cases	Pros	Cons
K-Fold CV	General use for i.i.d. data (independent and identically distributed)	Simple to implement Efficient use of data for training and validation	Not suitable for time-series data; Risk of unbalanced splits (if used naively for classification)
Stratified K-Fold	Classification with imbalanced classes	Preserves class distribution in each fold; Reduces bias from uneven class splits	Not directly suitable for regression or multilabel data (requires binning or custom methods)
Repeated K-Fold	Small datasets Need more robust estimates	Reduces variance in estimates by averaging multiple runs; Mitigates reliance on a single random split	Higher computational cost; Not time-series friendly
Time Series CV	Forecasting Chronological datasets	Maintains temporal ordering; Avoids data leakage from future	Reduced training data in each fold; Often more folds needed for stable estimates
Leave-One-Out CV	Very small datasets Comprehensive assessment	Maximum training data usage per fold; Thorough performance measure	Computationally expensive (one training run per sample); High variance for large datasets
Nested CV	Model comparison Hyperparameter tuning	Unbiased performance estimate; Separates tuning (inner loop) from final eval (outer loop)	Doubly expensive (CV within CV); More complex to implement and interpret



Hyperparameter Optimization

What is Hyperparameter Optimization

Hyperparameters

- Configurable choices that control **how** a model is trained (e.g., depth of a decision tree, learning rate for gradient-based methods).
- **Not** learned from data; they must be set manually or tuned.

Why They Matter

- **Directly** influence model performance and generalization.
- Prevent reliance on **default or arbitrary** settings, which may be suboptimal for your specific data.

Hyperparameter Optimization

- The **process** of systematically searching for the best hyperparameter settings.
- Aiming to **maximize** (or minimize) a chosen metric (e.g., accuracy, MSE).

Common Approaches

Grid Search

- Tries every combination in a predefined parameter grid.
- **Pros:** Simple, easy to interpret.
- **Cons:** Quickly becomes expensive as the number of parameters grows (curse of dimensionality).

Randomized Search

- Samples parameter combinations from specified distributions.
- **Pros:** More efficient, can discover good regions in large search spaces.
- **Cons:** Less exhaustive coverage than grid search; might miss optimal combos by chance.

Grid Search vs Random Search Illustration

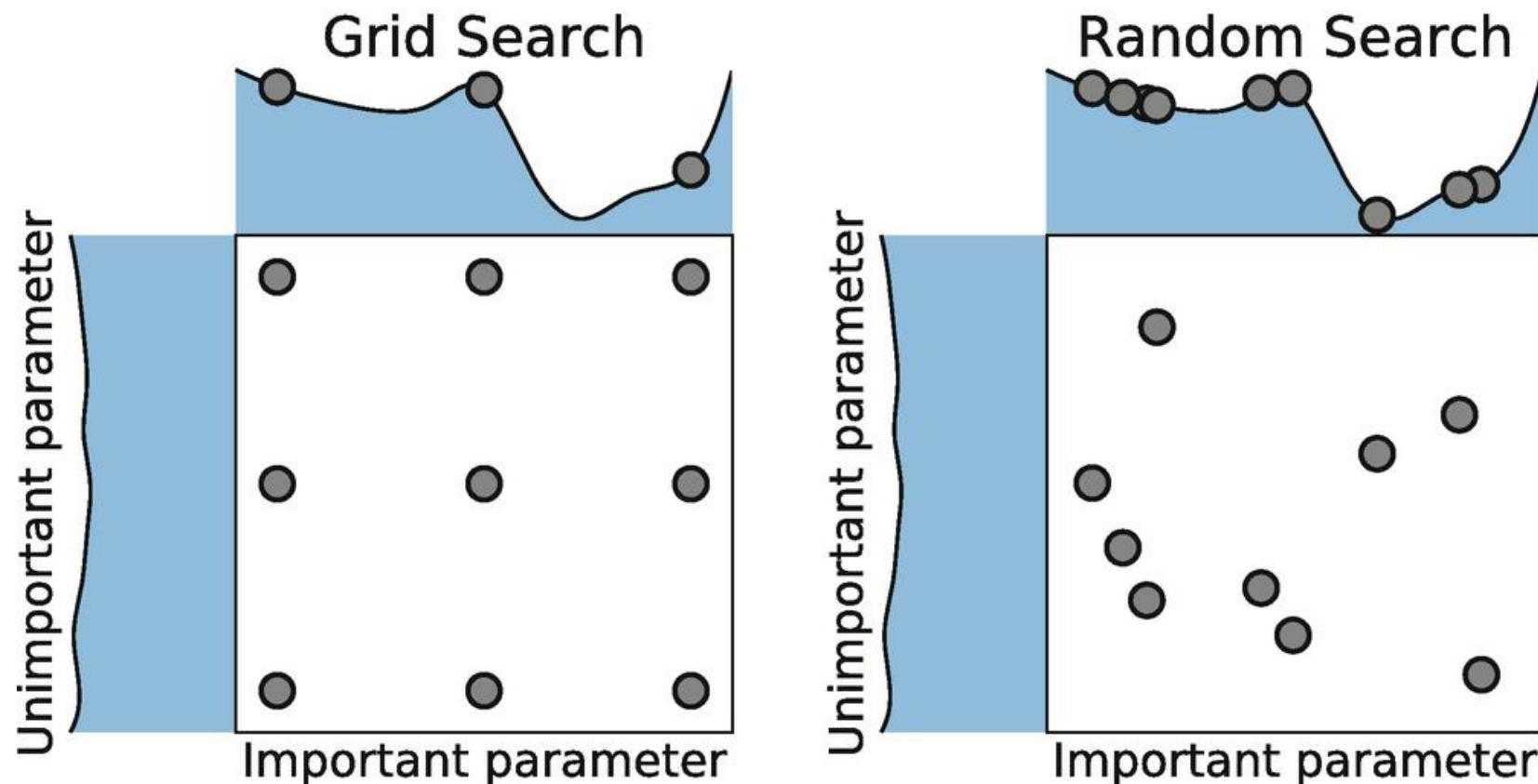


Image Source: Bergstra, J., Bengio, Y.: Random search for hyper-parameter optimization. Journal of Machine Learning Research 13, 281–305 (2012)

Bayesian Optimization

Builds a probabilistic model (e.g., Gaussian Process) of the objective function. Suggests next hyperparameter set based on past performance.

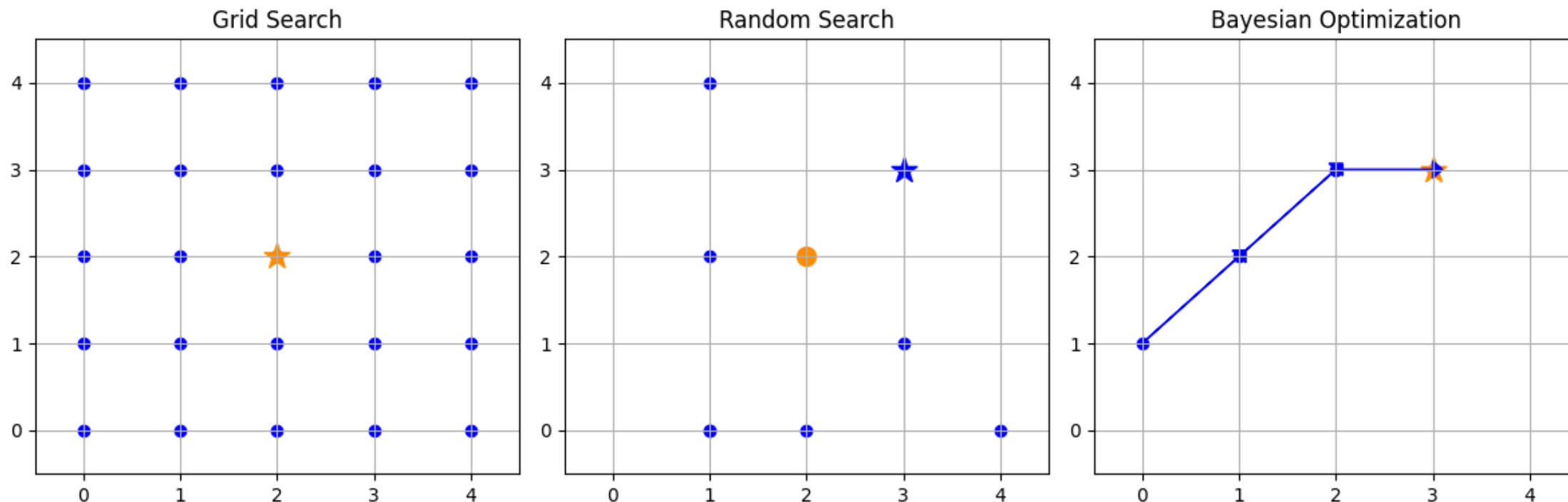
Popular Libraries

- Optuna, Hyperopt, scikit-optimize.

Pros/Cons

- **Pros:** More sample-efficient than random or grid search; can converge on good solutions faster.
- **Cons:** Increased complexity; more difficult to interpret than simpler methods.

Comparison of different optimization approaches



- Evaluation points
- ★ Optimal parameters
- ★ Local optimal parameters

Tools for Hyperparameter optimization

- **scikit-learn:**

- [GridSearchCV](#) and [RandomizedSearchCV](#)

- Integrates with Pipelines for seamless end-to-end tuning

- **[Optuna](#):**

- User-friendly Bayesian optimization API
 - Advanced features: hyperparameter pruning, visualization

- **[Ray Tune](#):**

- Scalable tuning framework for clusters or multicore systems
 - Integrates with various ML libraries

- **[Hyperopt](#):**

- Implements Tree-structured Parzen Estimator (TPE) for Bayesian optimization.
 - Good balance of simplicity and power

Use-cases

- **Fraud Detection (Finance & E-Commerce)**

Tuning tree-based models (e.g., Random Forest, GBM) to minimize false positives while catching more fraudulent transactions.

- **Predictive Maintenance (Manufacturing)**

Fine-tuning algorithms (e.g., SVM, Gradient Boosting) for more accurate fault detection from sensor data, reducing downtime and costs.

- **Recommendation Systems (Retail & Media)**

Adjusting hyperparameters in collaborative filtering or matrix factorization to improve the relevance of suggested products or content.

- **Marketing & Customer Retention**

Optimizing parameters in churn-prediction models (Logistic Regression, Decision Trees) to identify at-risk customers more precisely.

- **Forecasting & Demand Planning**

Tuning parameters in traditional time-series models (ARIMA, Prophet) for better inventory control, staffing, or resource allocation.

Practical Tips for Hyperparameter Selection

- **Identify Key Hyperparameters**

Not all parameters matter equally (e.g., learning rate vs. minor settings).

- **Progressive Tuning**

Start broad, then refine your search space.

Use coarse-to-fine grid or random search in stages.

- **Use Proper Validation**

Integrate hyperparameter search with cross-validation.

Avoid overfitting by testing the final choice on a separate test set.

- **Log Experiments**

Tools like MLflow or Weights & Biases help track experiments.

- **Mind your budget!**

Time, compute, and data constraints factor into tuning strategies.

- **Use Domain Knowledge!**

Prioritizing relevant parameters saves time and boosts performance.

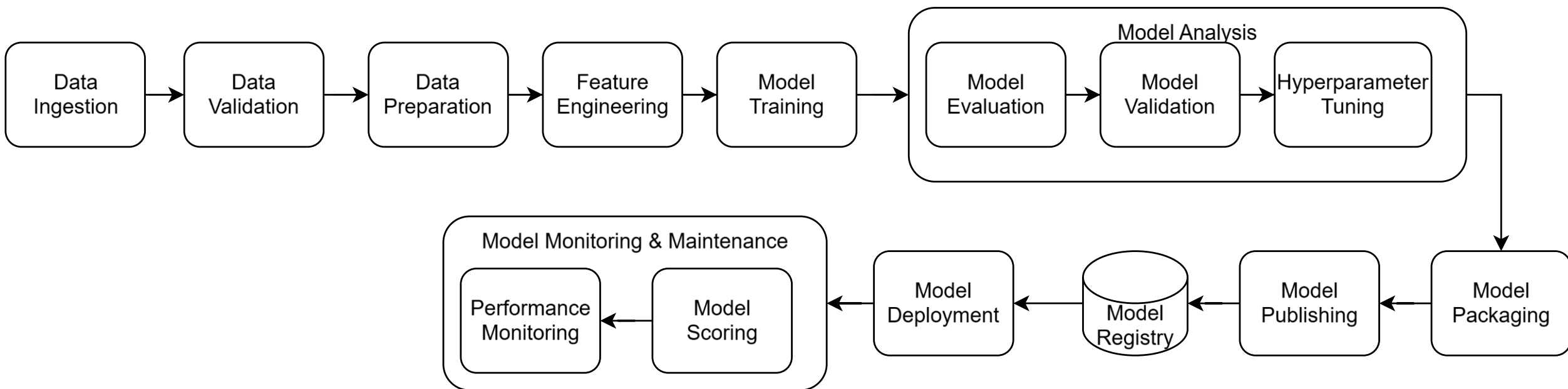
Machine Learning Pipelines

Feature Engineering

Deployment

Feature

Components of a Typical ML Pipeline



Why Use Pipelines?

Reproducibility:

Each step is clearly defined and can be rerun identically.

Automation:

Reduces manual effort and eliminates errors.

Modularity:

Easy to debug or replace individual components.

Scalability:

Integrates well with big data tools and frameworks.

Consistency:

Ensures consistent preprocessing during training and inference.

Example of scikit-learn pipeline

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split, GridSearchCV
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.pipeline import Pipeline
5 from sklearn.ensemble import RandomForestClassifier
6
7 X, y = load_iris(return_X_y=True)
8 X_train, X_test, y_train, y_test = train_test_split(X, y)
9
10 pipeline = Pipeline([
11     ("scaler", StandardScaler()),
12     ("clf", RandomForestClassifier())
13 ])
14
15 params = {"clf__n_estimators": [50, 100]}
16 grid = GridSearchCV(pipeline, params).fit(X_train, y_train)
17 print("Best parameters:", grid.best_params_)
18 print("Test score:", grid.score(X_test, y_test))
```

Key Tools for Classical ML Pipelines

1. Scikit-learn Pipelines

Why Use It?

Perfect for small- to medium-scale ML projects. Supports preprocessing, feature selection, model training, and evaluation in a single pipeline.

Key Features:

Integrates seamlessly with transformers like StandardScaler, PCA, and estimators like RandomForestClassifier.

```
1
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.feature_selection import SelectKBest
5 from sklearn.ensemble import RandomForestClassifier
6
7 pipeline = Pipeline([
8     ('scaler', StandardScaler()),
9     ('feature_selection', SelectKBest(k=10)),
10    ('classifier', RandomForestClassifier())
11 ])
12 pipeline.fit(X_train, y_train)
13 predictions = pipeline.predict(X_test)
14
```

Key Tools for Classical ML Pipelines

2. ML Flow

Why Use It?

Track experiments, compare runs, and manage models for classical ML workflows.

Key Features:

Logs parameters, metrics, and artifacts (like saved models).

Easily integrates with tools like scikit-learn.

```
1 import mlflow
2 import mlflow.sklearn
3 from sklearn.ensemble import RandomForestClassifier
4
5 with mlflow.start_run():
6     model = RandomForestClassifier(n_estimators=100)
7     model.fit(X_train, y_train)
8     mlflow.sklearn.log_model(model, "model")
9     mlflow.log_param("n_estimators", 100)
10    mlflow.log_metric("accuracy", model.score(X_test, y_test))
11
```


Key Tools for Classical ML Pipelines

3. Apache Airflow

Why Use It?

Orchestrates ML workflows (e.g., data preprocessing, model training, evaluation) using Directed Acyclic Graphs (DAGs).

Key Features:

Schedule and monitor pipelines.

Automates preprocessing, training, and evaluation steps.

```
1 from airflow import DAG
2 from airflow.operators.python_operator import PythonOperator
3
4 def preprocess_data():
5     # Preprocessing code here
6     pass
7
8 def train_model():
9     # Training code here
10    pass
11
12 dag = DAG('ml_pipeline', schedule_interval='@daily')
13 preprocess_task = PythonOperator(task_id='preprocess', python_callable=preprocess_data,
14                                 dag=dag)
15 train_task = PythonOperator(task_id='train', python_callable=train_model, dag=dag)
16 preprocess_task >> train_task
```

AutoML

AUTOMATED
AUTOEML

BEST
MODEL

BEST MODEL

Introduction to AutoML

AutoML = Automated Machine Learning

Automates end-to-end ML processes.

Why AutoML?

- **For non-experts:** Simplifies ML workflows.
- **Saves time:** Automates tedious tasks.
- **Improves performance:** Systematically explores models.

Tools and Solutions for AutoML

Popular Frameworks:

- [H2O AutoML](#)
- [Auto-sklearn](#)
- [TPOT](#): Tree-based Pipeline Optimization Tool
- [MLJAR AutoML](#)
- [LightAutoML](#)
- [Etna](#)
- Cloud Platforms:
 - [Google Cloud AutoML](#)
 - [AWS SageMaker Autopilot](#)
 - [Azure AutoML](#)
 - [DataRobot](#)
- Pros & Cons:
 - **Pros:** Can save time, find novel pipelines, great for baselines
 - **Cons:** Computationally heavy, less interpretability, sometimes black-box

Simple AutoML Code – H2O AutoML

```

1 # Import necessary libraries
2 import h2o
3 from h2o.automl import H2OAutoML
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6 import pandas as pd
7
8 # Initialize H2O
9 h2o.init()
10
11 # Load dataset (Iris dataset for demonstration)
12 data = load_iris()
13 X, y = pd.DataFrame(data.data, columns=data.feature_names), pd.DataFrame(data.target,
14 columns=["target"])
15
16 # Split dataset into training and testing sets
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
18 random_state=42)
19
20 # Combine training data into a single H2OFrame
21 train = h2o.H2OFrame(pd.concat([X_train, y_train], axis=1))
22 test = h2o.H2OFrame(pd.concat([X_test, y_test], axis=1))
23
24 # Set target and predictors
25 target = "target"
26 predictors = data.feature_names
27
28 # Run H2O AutoML
29 aml = H2OAutoML(max_runtime_secs=120, seed=42) # Run for 2 minutes
30 aml.train(x=predictors, y=target, training_frame=train)
31
32 # View leaderboard
33 lb = aml.leaderboard
34 print(lb)
35
36 # Make predictions on the test set
37 predictions = aml.leader.predict(test)
38 print(predictions)
39
40 # Evaluate model performance
41 perf = aml.leader.model_performance(test)
42 print(perf)
43

```

Parse progress:  (done) 100%

Parse progress:  (done) 100%

AutoML progress: 

23:46:04.884: _min_rows param, The dataset size is too small to split for min_rows=100.0: must have at least 200.0 (weighted) r

 (done) 100%

model_id	rmse	mse	mae	rmsle	mean_residual_deviance
DeepLearning_grid_1_AutoML_1_20250105_234559_model_1	0.223747	0.0500627	0.13478	0.100144	0.0500627
StackedEnsemble_BestOfFamily_2_AutoML_1_20250105_234559	0.227939	0.0519563	0.145362	0.0970081	0.0519563
DeepLearning_grid_1_AutoML_1_20250105_234559_model_2	0.228703	0.0523051	0.172824	0.103104	0.0523051
GBM_grid_1_AutoML_1_20250105_234559_model_2	0.231225	0.053465	0.103157	0.0939578	0.053465
GBM_grid_1_AutoML_1_20250105_234559_model_10	0.235722	0.0555647	0.134223	0.102468	0.0555647
GLM_1_AutoML_1_20250105_234559	0.236152	0.0557677	0.182473	0.110809	0.0557677
DeepLearning_grid_3_AutoML_1_20250105_234559_model_1	0.236235	0.0558068	0.166211	0.107902	0.0558068
GBM_grid_1_AutoML_1_20250105_234559_model_32	0.236827	0.0560868	0.127334	0.0992283	0.0560868
StackedEnsemble_BestOfFamily_1_AutoML_1_20250105_234559	0.237888	0.0565909	0.15616	0.101674	0.0565909
GBM_2_AutoML_1_20250105_234559	0.238994	0.0571183	0.125262	0.102137	0.0571183

[100 rows x 6 columns]

deeplearning prediction progress:  (done) 100%

predict
1.27862
0.0321074
2.00057
1.09812
1.11829
-0.0120915
0.954327
2.10908
1.3246
1.02635
[45 rows x 1 column]

ModelMetricsRegression: deeplearning

** Reported on test data. **

MSE: 0.01127066226009239

RMSE: 0.10616337532356622

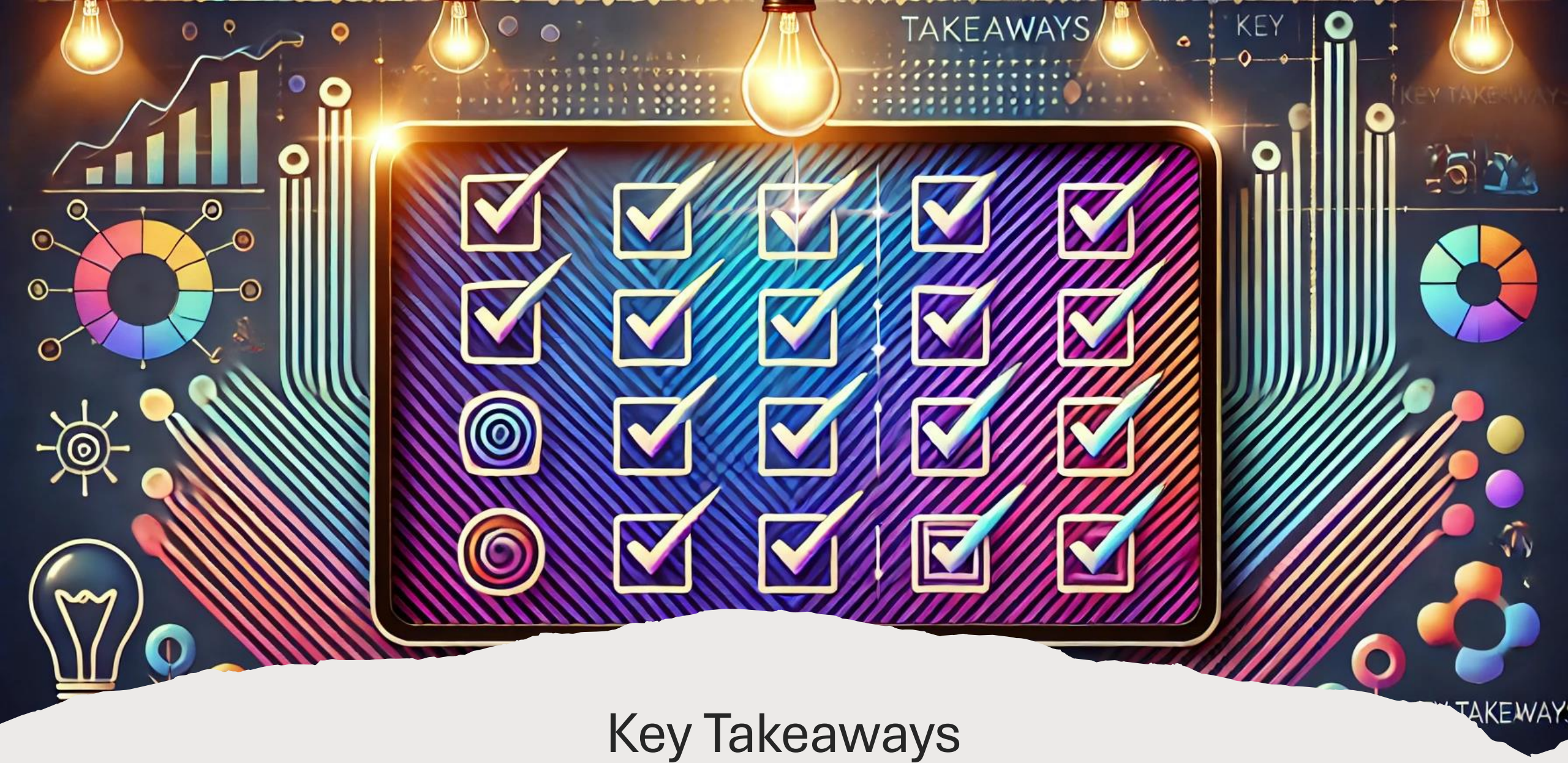
MAE: 0.07279617598295923

RMSLE: 0.04815101497145974

Mean Residual Deviance: 0.01127066226009239

Live Coding & Demos





Key Takeaways

Practical Tips & Best Practices

- **Keep an Independent Test Set:** Always hold out final data to verify your best pipeline
- **Experiment Tracking:** Tools like MLflow or TensorBoard for logging
- **Compute Resources:** Parallelize if possible, or use cloud-based solutions
- **Interpretability:** Complex pipelines or AutoML outcomes can be harder to interpret

Key Takeaways

- **Cross-Validation:** Essential for robust performance metrics
- **Hyperparameter Tuning:** GridSearch, RandomizedSearch, Bayesian Optimization
- **Pipelines:** Integrate transformations and models in a single workflow
- **AutoML:** Automated pipeline and hyperparameter selection; be mindful of resource usage

Thank you for
your attention

